# 실전 웹서비스개발 – 6주차

웹 서비스 인증과 보안

# 학습 목표

- 웹 서비스에서 인증의 필요성
- 웹 서비스 인증의 흐름과 종류
- 웹 서비스 운영
- 웹 서비스 인증 시스템 구현

# 웹 서비스에서 인증이 필요한 이유

## 상태 유지

- HTTP는 무상태 → 사용자 식별 필요

## 보안 강화

- 권한이 없는 기능과 데이터 접근 차단

## 권한 제어

- 역할(Role)별 접근 제한

## 개인화

- 사용자 맞춤 데이터

## 감사 로그

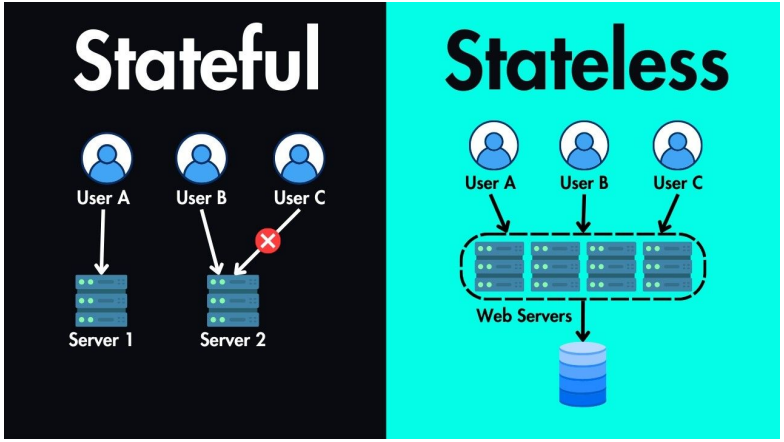
- 추적 가능성 확보

# 상태 유지

## HTTP 프로토콜의 무상태(stateless)성

- HTTP 프로토콜은 요청(Request)과 응답(Response)이 끝나면 연결 정보를 기억하지 않음
- 서버는 “이 요청이 누구로부터 왔는지”를 자동으로 알 수 없음
- 사용자가 로그인하더라도, 다음 요청 시 “이전 로그인 정보”가 사라짐

  → 인증 토큰이나 세션을 이용해 사용자를 식별



# 보안(Security)

- 인증이 없으면 누구나 임의로 데이터에 접근 가능
- 게시물 삭제, 결제 정보, 개인정보 노출 등의 위험 발생

  → 사용자가 누구인지 확인하는 절차가 반드시 필요



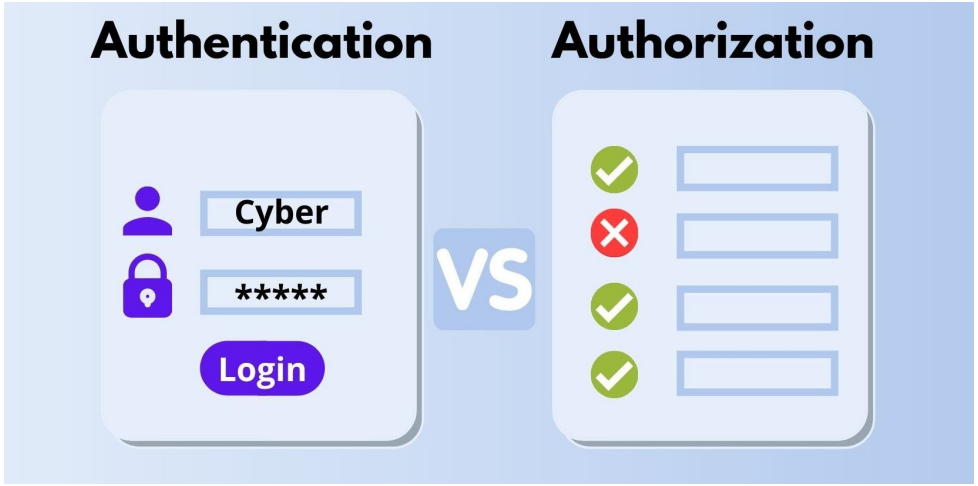
# 권한(Authorization) 제어

## 서비스는 사용자마다 권한이 다름

- **인증(Authentication)**: “누구인가?”
- **권한(Authorization)**: “무엇을 할 수 있는가?”

Ex)

- **일반 사용자**
  - 맛집 제보
  - 맛집 조회
- **관리자**
  - 맛집 인증, 거절
  - 맛집 생성, 조회, 수정, 삭제

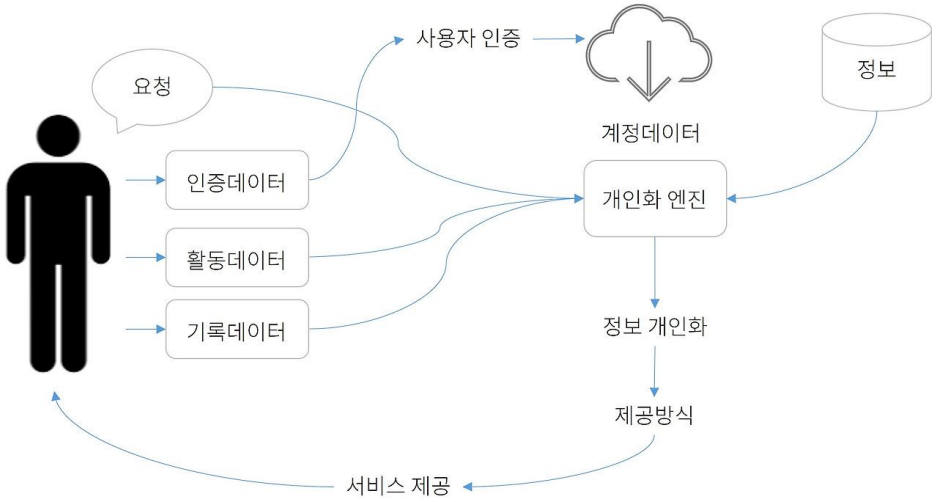


# 개인화 서비스(Personalization Service)

로그인한 사용자를 식별해

- 내 피드
- 내 주문내역
- 내 설정

→ **인증**은 사용자별 맞춤 데이터 제공의 출발점



# 트래킹 및 감사 로그(Audit Log)

- 누가 언제 어떤 행동을 했는지 기록
- 보안 사고 대응, 이상 행동 탐지



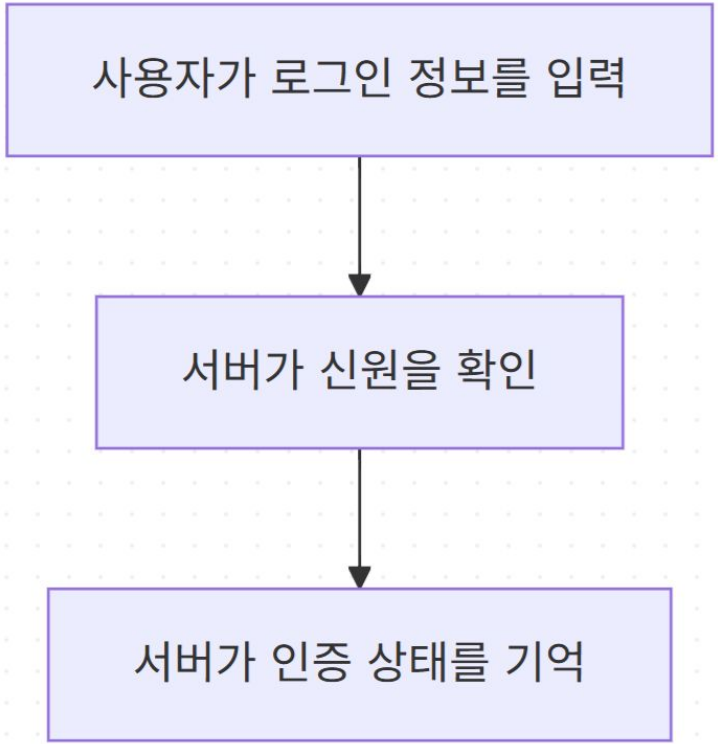
# 웹 서비스 인증의 흐름

사용자는 아이디(ID)와 비밀번호(Password) 같은 자격 증명을 클라이언트(브라우저나 앱)에서 입력




서버는 전달받은 로그인 정보를 DB에 저장된 사용자 정보와 비교
→ 일치하면 "인증 성공", 불일치하면 "인증 실패"

인증에 성공하면 서버는 사용자가 로그인한 상태임을 기억하기 위해 세션, 쿠키, JWT 등을 이용해 인증 정보를 저장
→ 이후 요청에서는 이 인증 정보를 통해 사용자를 식별



# 인증의 종류

## 인증 정보를 저장·유지하는 방식에 따라

- 세션 기반 인증 (Session-based Authentication)
- 토큰 기반 인증 (Token-based Authentication)
- OAuth 2.0 기반 인증 (Third-party Authentication)

# 세션 기반 인증 (Session-based Authentication)

## 세션 ID를 쿠키에 담아 전송하여 사용자 식별

### 저장 위치

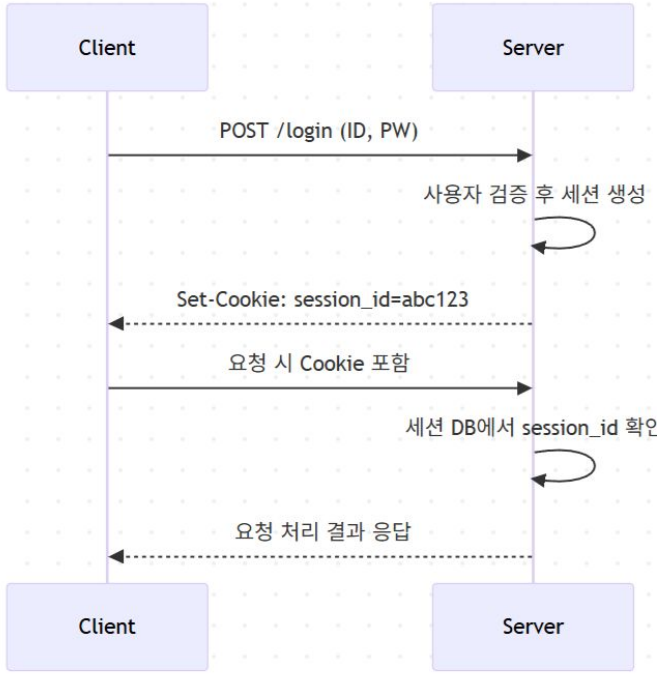
- 서버 메모리(DB)에 세션 정보 저장

### 대표 사용 사례

- 전통적인 웹 (PHP, JSP, Express+EJS 등)

### 특징

- 장점: 단순, 안정적, 서버가 모든 상태를 관리
- 단점: 서버 확장(스케일아웃)에 불리 → 세션 동기화 필요
- 사용 예시: 로그인 유지, 관리자 페이지



# 쿠키(Cookie)

## HTTP 무상태성을 보완하기 위해 브라우저가 도메인별로 저장하는 작은 문자열

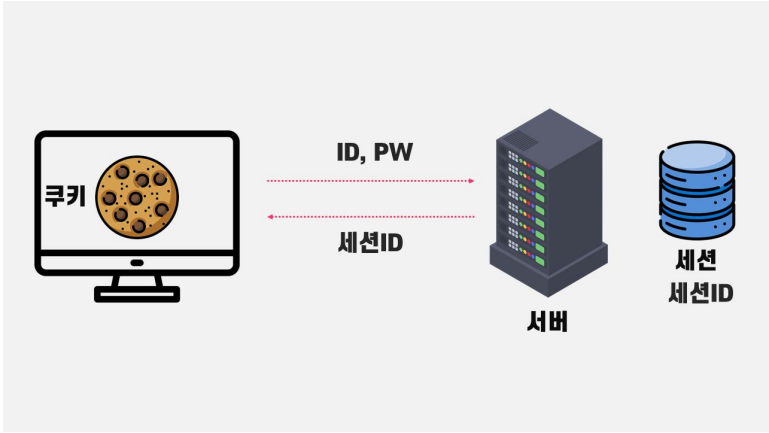
- 서버가 Set-Cookie 헤더로 내려주면 브라우저가 저장
- 같은 도메인으로의 요청에 자동으로 Cookie 헤더로 보냄
- 세션 식별, 로그인 유지, 트래킹/설정 저장에 사용
- 크기 제한(관례): 쿠키 1개 ≈ 4KB 내외, 도메인당 수십~수백 개(브라우저별 상이).

```
Set-Cookie: name=value; Expires=Tue, 21 Jan 2030 07:28:00 GMT; Path=/; Domain=example.com; Secure; HttpOnly;
SameSite=Lax
```

# 세션(Session)

## HTTP의 무상태성 때문에 사용자 상태를 서버에 저장

- 브라우저에는 상태를 찾기 위한 키(세션ID)만 쿠키로 보관.
- 서버 상태 저장형 인증




## 구성요소

- **세션 스토어**
  - 세션 데이터 저장소(메모리, MongoDB 등)
  - 서버 재시작/스케일아웃 대비하려면 외부 스토어 권장
- **세션 객체**
  - { userId, role, cart, ... } 같은 사용자 상태
  - 스토어에 보관, 서버에서만 읽기/수정
- **세션 ID**
  - 난수/암호학적 토큰 (예: sid=abc123...)
  - 브라우저 쿠키에 저장(HttpOnly; Secure)

### 세션 설정

```
const sessionConfig = {
    secret: process.env.SESSION_SECRET || 'your-secret-key-change-in-production',
    resave: false,
    saveUninitialized: false,
    cookie: {
      maxAge: 1000 * 60 * 60 * 24 * 7, // 7일
      httpOnly: true,
      secure: process.env.NODE_ENV === 'production', // HTTPS에서만 쿠키 전송
      sameSite: process.env.NODE_ENV === 'production' ? 'none' : 'lax',
    },
  };
```

# 토큰 기반 인증 (Token-based Authentication)

## 토큰을 인증(Authorization) 헤더로 전송하여 사용자 식별

### 저장 위치

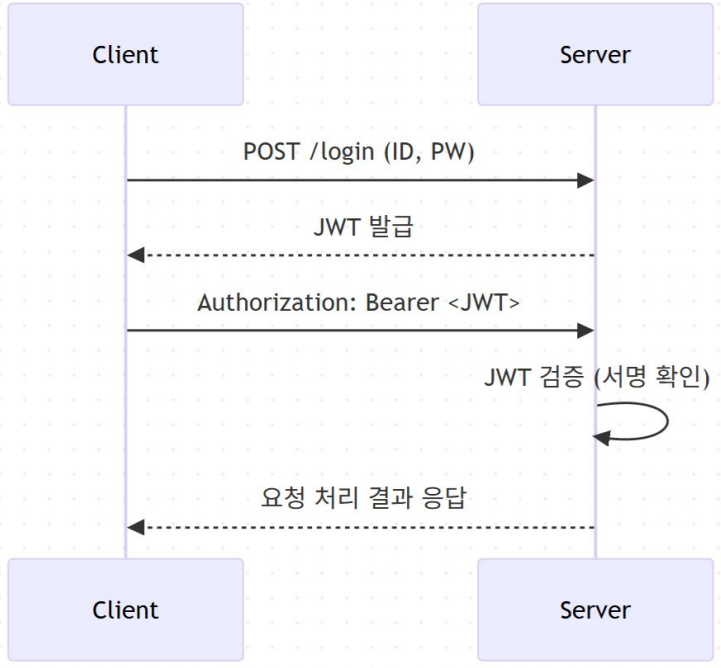
- 클라이언트(브라우저/앱) 로컬 저장소

### 대표 사용 사례

- SPA, 모바일 앱, REST API 서버

### 특징

- **장점**
  - 서버 상태를 저장하지 않아 수평 확장에 유리
  - 다양한 클라이언트(웹/앱)에서 사용 가능
- **단점**
  - 토큰 탈취 시 위험
  - 만료시간·재발급 로직 필요



# JWT(JSON Web Token)

## 서버 상태를 저장하지 않고(stateless) 사용자 신원을 토큰 자체에 담아 전달하는 방식

- 문자열 형태 (각 부분은 Base64URL 인코딩)

## 토큰 구조




- **Header**
  - 서명 알고리즘(HS256/RS256 등), 타입
  - {"alg":"HS256","typ":"JWT"}
- **Payload**
  - 사용자·권한·만료 등 정보
  - {"sub":"u123","role":"user","iat":...,"exp":...}
- **Signature**
  - 위변조 방지를 위한 서명
  - HMACSHA256(base64Url(header)+"."+base64Url(payload), secret)

# 세션 vs JWT

| 항목 | 세션(Session) | JWT(Stateless) |
|---|---|---|
| **상태 저장** | 서버 스토어 필요 | 불필요 |
| **스케일** | 스토어 공유 필요 | 수평 확장 유리 |
| **무효화** | 스토어에서 즉시 삭제 | 블랙리스트/짧은 만료/버전필드 필요 |
| **노출 시 위험** | 세션ID만 노출 | 사용자 정보·권한 서버 노출 가능 |
| **적합** | 전통 웹/내부 | SPA/모바일/API 게이트웨이 |

# OAuth 2.0 기반 인증 (Third-party Authentication)

## OAuth Provider가 발급한 Access Token으로 사용자 인증

### 저장 위치

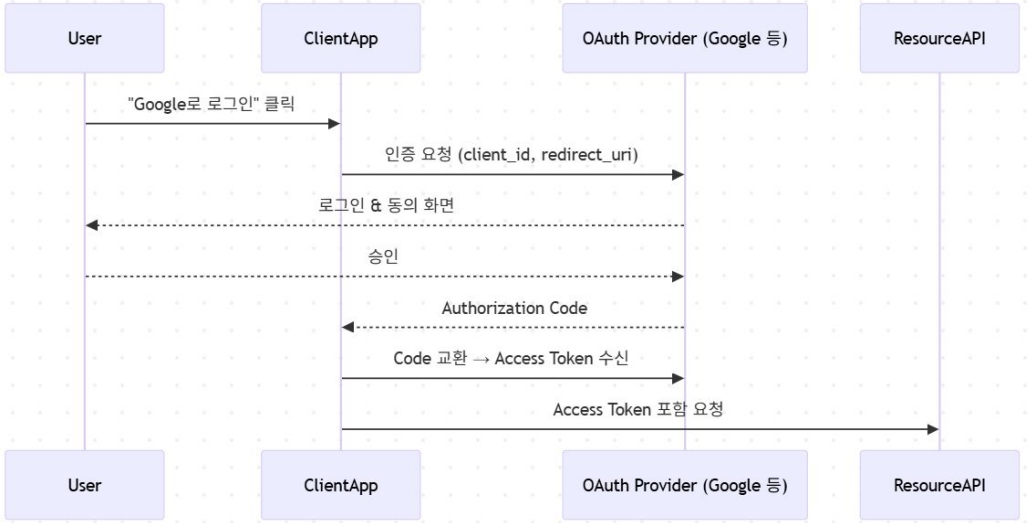
- 외부 인증 서버(Google, Naver, Kakao 등)

### 대표 사용 사례

- 소셜 로그인, API 연동 서비스

### 특징

- 장점
  - 비밀번호 저장 불필요 (보안 ↑)
  - 다양한 플랫폼과 계정 연동 가능
- 단점
  - 구현 구조 복잡, 리다이렉트·토큰 관리 필요
- 사용 예시
  - Google, Apple, Facebook, Naver, Kakao 로그인



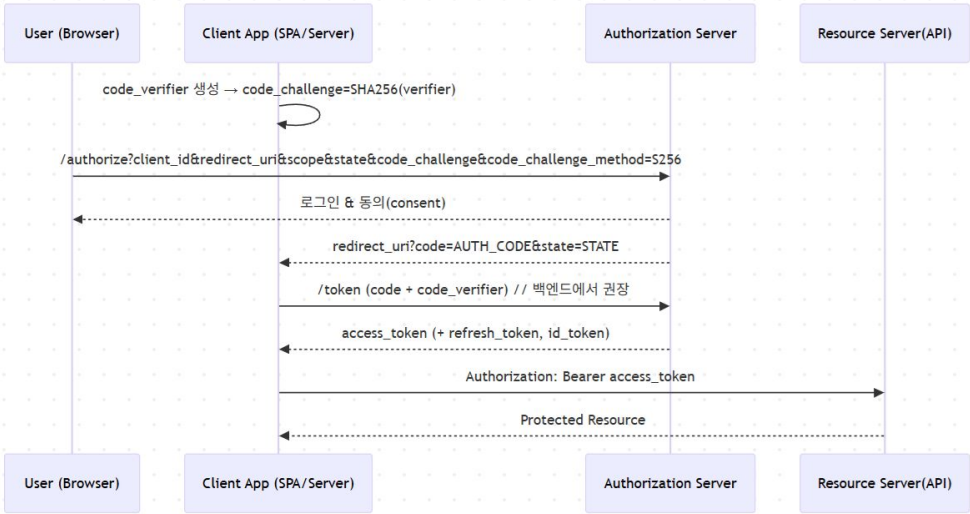
# OAuth(Open Authorization) 2.0

## OAuth 2.0(Open Authorization 2.0, OAuth2)은 인증을 위한 개방형 표준 프로토콜

- Third-Party 프로그램에게 리소스 소유자를 대신하여 리소스 서버에서 제공하는 자원에 대한 접근 권한을 위임하는 방식

### 주요 용어

- **client_id / client_secret**
  - 클라이언트 자격증명.
  - 서버에 등록 시 발급받아 인증에 사용
- **redirect_uri**
  - 서버가 인증 결과(code/token)를 보낼 콜백 URL
- response_type
  - 서버 부여 방식 지정
- expires_in
  - 토큰 유효시간(초 단위)



# 웹서비스 보안과 운영

# 일반적인 웹 공격

## 스크립트 주입(Cross-Site Scripting, XSS)

- 웹페이지에 악성 JavaScript 코드를 삽입해 다른 사용자의 브라우저에서 그 코드를 실행
- XSS로 발생할 수 있는 피해
  - 세션 쿠키나 JWT 토큰 탈취
  - 사용자 계정 도용
  - 악성 스크립트 삽입으로 피싱 페이지 전환
  - 관리자 계정 탈취를 통한 데이터 유출

## 요청 위조(Cross-Site Request Forgery, CSRF)

- 사용자가 이미 로그인된 상태를 악용해 공격자가 의도한 요청을 사용자의 브라우저에서 몰래 실행하도록 만드는 공격

## SQL Injection

- 공격자가 입력값에 악성 SQL문을 삽입해 애플리케이션의 데이터베이스 쿼리를 조작·조회·변경하거나 권한을 탈취하는 공격

## Brute Force 공격

- 공격자가 자동화 도구로 가능한 비밀번호를 무작위·순차적으로 대입해 계정에 무단 접근을 시도하는 공격

## Directory Traversal

- 경로 조작을 이용해 서버 파일 시스템의 의도치 않은 파일(예: /etc/passwd, 비밀 키 파일)을 읽거나 접근하는 공격

# 환경 설정 및 비밀정보 관리

## 환경변수(.env)

- API Key, JWT Secret, OAuth Client Secret 등 저장
- 코드에 직접 작성 금지 (.gitignore 필수)

## GitHub 공개 저장소 주의

- .env나 토큰이 푸시되면 즉시 폐기
- 과거 커밋에도 남을 수 있음 → 재발급 필요

## 배포 환경 분리

- dev/stage/prod 환경별 Secret 구분
- 동일 키를 모든 환경에서 사용하지 않기

# 인증 토큰 관리(JWT, Access Token 등)

## 짧은 만료시간 설정

- Access Token은 15~60분 이내 권장 (금융 서비스 10 ~ 15분)
- Refresh Token 관리
  - 더 길게(예: 수일~수주) 유지하되, 안전하게 보관·회수 가능해야 함

## 토큰 저장 위치

- 가능하면 HttpOnly + Secure Cookie 사용(JS에서 접근 불가)
- localStorage는  악성 스크립트에 취약

## 토큰 탈취 방지

- HTTPS 필수, 토큰 로그 출력 금지
- 콘솔/네트워크탭에 남지 않도록

## 로그아웃 처리

- 토큰 블랙리스트 또는 만료로 무효화
- 클라이언트에서만 삭제 X

# 세션 및 쿠키 설정

## 쿠키 속성 설정

- JS 접근 차단, HTTPS만 전송
- Secure : 플래그가 설정된 쿠키는 HTTPS(암호화된 연결) 일 때만 브라우저가 서버로 전송
  - 네트워크에서 트래픽을 가로채는 공격자(중간자, MITM)가 쿠키를 훔치는 걸 막아줌
  - 특히 인증 쿠키나 토큰을 HTTP로 보내면 심각한 계정 탈취 위험이 있음
- HttpOnly : 플래그가 있으면 브라우저의 JavaScript에서 해당 쿠키에 접근(읽기/쓰기) 불가
  - XSS 공격이 성공하더라도, 공격 스크립트가 쿠키(예: 세션ID, 리프레시 토큰)를 직접 읽어 서버로 전송하는 것을 막음
  - XSS로 인한 토큰탈취 위험을 크게 줄이는 핵심 수단
- SameSite : 교차 사이트 요청(cross-site requests) 시 브라우저에서 전송될지 여부를 제어
  - SameSite=Strict - 동일 사이트(same-site) 요청에서만 쿠키 전송
  - SameSite=Lax - 위 수준 네비게이션(GET 링크 클릭, 주소창 진입) 같은 “안전한” 탐색에서는 쿠키 전송 허용
  - SameSite=Non - 교차 사이트에서도 항상 전송

## 세션 스토리지 관리

- 외부 스토어 권장
- 서버 재시작 시 세션 손실 방지

## 자동 로그아웃 (Timeout)

- 장기 미사용 세션 종료
- 일정 시간 후 세션 만료 처리

# 로그와 모니터링

## 민감정보 로그 금지

- 비밀번호, 토큰, 이메일 전체 로그 X
- 마스크 처리

## 보안 이벤트 모니터링

- 로그인 실패, 권한 오류 등 기록
- 이상 패턴 탐지(Alert)

## 감사 로그 분리 저장

- 운영 로그와 분리, 장기 보존
- AWS CloudWatch, ELK Stack 등 활용

# 실습

# Passport.js

## Node.js/Express용 인증 미들웨어

- 다양한 인증 관련 기능(로컬, OAuth, JWT 등)을 플러그인 형태로 연결해 로그인 흐름을 쉽게 구현 가능

## 핵심 개념

- **Strategy**: 인증 방식별 로직 구현 (예: passport-local, passport-github, passport-jwt)
- **Middleware**: Express 미들웨어처럼 app.use(passport.initialize()) 등으로 사용
- **serializeUser / deserializeUser**: 세션 기반일 때 사용자 객체를 세션에 저장/복원하는 함수
- **유연성**: 플러그인 방식이라 로컬 로그인, OAuth(구글/깃허브), 토큰 기반(JWT) 모두 가능

## 장점

- 전략 분리로 구현 간결 → 여러 로그인 방식 동시 지원 쉬움
- 커뮤니티 전략이 많아(구현 재사용) 빠르게 연동 가능

# 국외 주요 OAuth 2.0 로그인 제공 서비스

## 국외

| 구분 | 서비스명 | 제공 방식 / 특징 |
|---|---|---|
| **Google** | Google OAuth 2.0 | Gmail, YouTube, Calendar 등과 통합 / 가장 보편적 |
| **GitHub** | GitHub OAuth Apps | 개발자 계정 로그인 / Access Token으로 사용자 Repo 접근 가능 |
| **Facebook (Meta)** | Facebook Login | SNS 프로필 기반 로그인 / Pixel·광고 연계 기능 |
| **Apple** | Sign in with Apple | 개인정보 최소화 / iOS, macOS 통합 로그인 |

## 국내

| 구분 | 서비스명 | 제공 방식 / 특징 |
|---|---|---|
| **NAVER** | 네이버 아이디로 로그인 | 대한민국에서 가장 많이 사용되는 OAuth / SNS·쇼핑 등 다목적 |
| **Kakao** | 카카오계정 로그인 | 카카오톡 연계 / 친구 목록·메시지 전송 등 API 제공 |

# Google

https://console.cloud.google.com/

# Naver

https://developers.naver.com/

# 과제 개요

## 캠퍼스 푸드맵 인증 구현 및 운영 환경 구축

### 학습 목표

- 웹 서비스에서 인증의 필요성
- 인증의 흐름과 종류
- 웹 서비스 운영
- 웹 서비스 인증 구현

### 필수 구현 사항

- 유저 유형 추가
- 유저 권한에 따른 기능 구현
- 구글 로그인
- 네이버 로그인

### 마감일

- 10월 19일 23시 59분까지

### GitHub URL(저장소 각각 제출)

- (예: https://github.com/***username***/pwd-week6-server)
- (예: https://github.com/***username***/pwd-week6-client)

### React App URL(Render 서버 연동)

- (예: https://pwd-week6-**[username]**.onrender.com)
- (예: https://pwd-week6-**[username]**.netlify.app)
- (예: https://pwd-week6-**[username]**.vercel.app)
- netlify, vercel 배포한 링크만 제출

# Q&A